

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу

«Операционные системы»

Группа: М8О-216БВ-24

Студент: Шитов А.С.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 05.11.25

Москва, 2025

Постановка задачи

Вариант 14.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы. В отчете привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков. Получившиеся результаты необходимо объяснить.

Есть набор 128 битных чисел, записанных в шестнадцатеричном представлении, хранящихся в файле. Необходимо посчитать их среднее арифметическое. Округлить результат до целых. Количество используемой оперативной памяти должно задаваться "ключом"

Общий метод и алгоритм решения

Использованные системные вызовы:

- `ssize_t write(int fd, const void* buf, size_t count)` – записывает данные из буфера в файловый дескриптор.
- `void exit(int status)` – завершает выполнение процесса с указанным статусом.
- `int clock_gettime(clockid_t clk_id, struct timespec* tp)` – получает время от указанных часов (`CLOCK_MONOTONIC`). Используется для измерения *wall-time* времени выполнения программы.
- `int pthread_create(pthread_t* thread, const pthread_attr_t* attr, void* (start_routine)(void*), void* arg)` – создает новый поток выполнения.
- `int pthread_join(pthread_t thread, void** thread_return)` – ожидает завершения указанного потока и получает его возвращаемое значение. Используется для синхронизации потоков и получения результатов.

Программа принимает три аргумента (ключа) командной строки: количество потоков, количество ОЗУ, путь к входному файлу. Выделяем память под массив, размер которого определяется переданным ключом ОЗУ. Затем программа считывает определенное количество чисел из файла и записывает их в выделенный массив.

Создаются и запускаются рабочие потоки, каждому из которых назначается определенная часть массива для обработки (нахождение частичной суммы чисел). Основной поток ожидает окончания вычислений с помощью `pthread_join` и суммирует частичные суммы, обновляет глобальную сумму. Затем программа считывает следующий блок чисел, записывает в массив и вычисления повторяются. По окончании всех вычислений программа освобождает всю выделенную память, выводит результат и завершает работу.

Данное решение демонстрирует эффективное использование многопоточности для распараллеливания задач.

Анализ ускорения и эффективности

Число потоков	Количество ОЗУ	Время исполнения (мс)	Ускорение	Эффективность
1	1048576	1342	1	1
2	1048576	752	1,78	0,89
4	1048576	455	2,95	0,74
8	1048576	307	4,37	0,55
16	1048576	284	4,73	0,30
32	1048576	302	4,44	0,14
100	1048576	339	3,96	0,04

Ускорение показывает во сколько раз применение параллельного алгоритма уменьшает время решения задачи по сравнению с применением последовательного алгоритма. Ускорение определяется величиной $S_N = T_1 / T_N$, где T_1 – время выполнения на одном потоке, T_N – время на N потоках.

Эффективность – величина $E_N = S_N / N$, где S_N – ускорение, N – количество используемых потоков

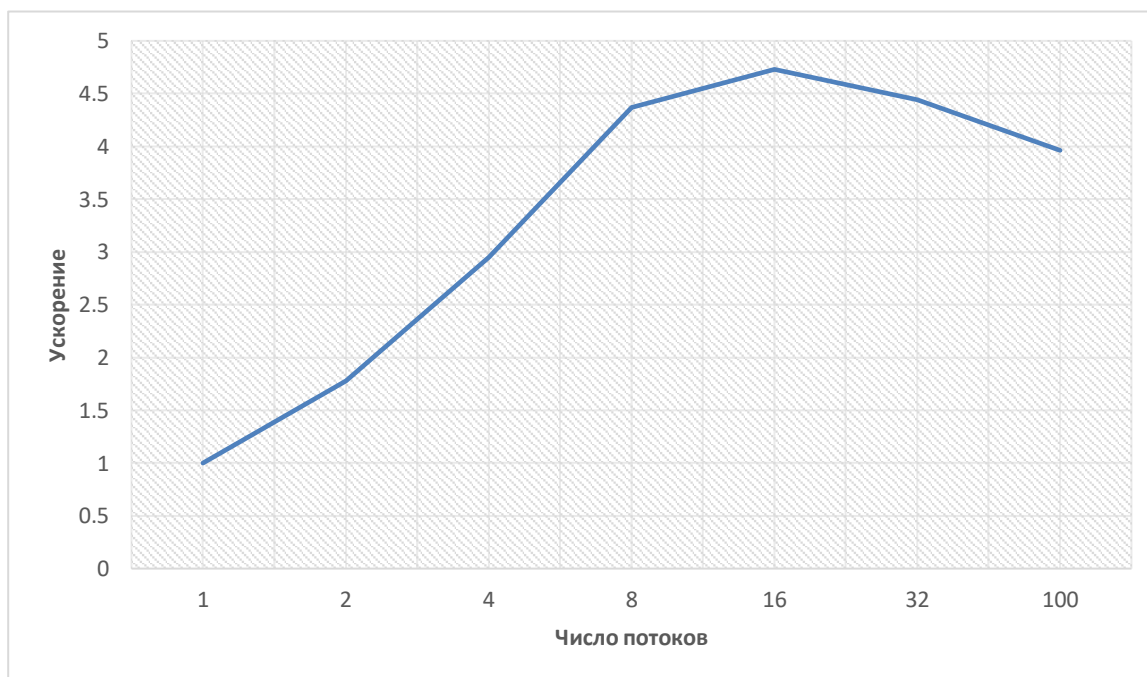


График показывает зависимость ускорения работы программы от числа потоков. При увеличении числа потоков с 1 до 8 ускорение быстро растет, дальнейшее увеличение числа потоков с 8 до 16 демонстрирует эффект «плато», когда увеличение числа потоков незначительно ускоряет программу, а последующие увеличения количества потоков приводят к замедлению работы программы, так как программе постоянно приходится переключать контексты между потоками.

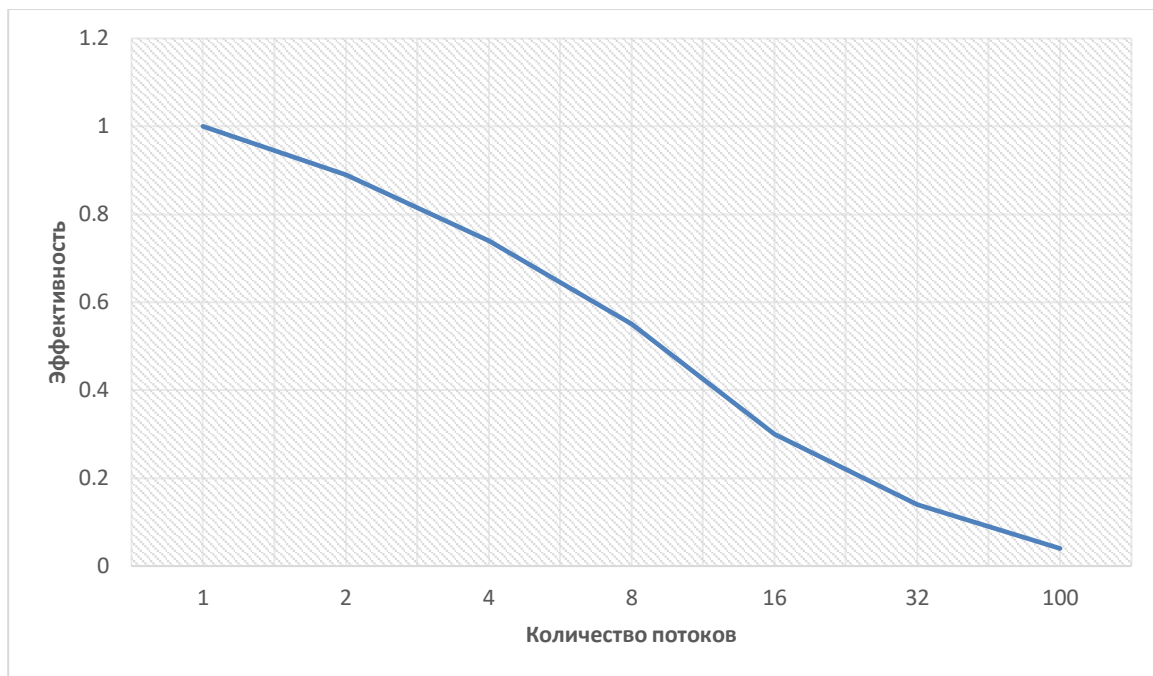


График показывает зависимость эффективности выполнения программы от числа потоков. При увеличении числа потоков с 1 до 8 эффективность убывает медленно, дальнейшее увеличение числа потоков демонстрирует стремительные потери эффективности, что соотносится с графиком ускорения программы от числа потоков.

Данные графики демонстрируют закон Амдала. Закон иллюстрирует ограничение роста производительности вычислительной системы с увеличением числа вычислителей.

Можно сделать вывод, что самым важным является нахождение баланса между ускорением и эффективностью использования ресурсов.

Код программы

hexFuncs.h

```
#ifndef HEXFUNCS_H
#define HEXFUNCS_H

#include <stdint.h>

char* hexSum(const char* n1, const char* n2);

char* hexDivide(const char* hex, uint64_t divisor);

#endif //HEXFUNCS_H
```

hexFuncs.c

```
#include "hexFuncs.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include <ctype.h>

char* hexSum(const char* n1, const char* n2)
{
    size_t len1 = strlen(n1);
    size_t len2 = strlen(n2);
    size_t maxLen = len1 > len2 ? len1 : len2;
    size_t resultLen = maxLen + 2;

    char *result = malloc(resultLen);
    if (!result)
    {
        char* msg = "Error: unable to allocate memory\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        return NULL;
    }

    uint8_t carry = 0;
    uint64_t writePos = resultLen - 2;
    result[resultLen - 1] = 0;

    for (size_t i = 0; i < maxLen; ++i)
    {
        uint8_t digit1 = 0, digit2 = 0;

        if (i < len1)
        {
            char c = n1[len1 - 1 - i];
            digit1 = isdigit(c) ? c - '0' : c - 'a' + 10;
        }

        if (i < len2)
        {
            char c = n2[len2 - 1 - i];
```

```

        digit2 = isdigit(c) ? c - '0' : c - 'a' + 10;
    }

    uint8_t sum = digit1 + digit2 + carry;
    carry = sum / 16;
    sum %= 16;

    result[writePos--] = sum < 10 ? sum + '0' : sum - 10 + 'a';
}

if (carry)
{
    result[writePos--] = carry < 10 ? carry + '0' : carry - 10 + 'a';
}

char *finalResult = malloc(strlen(result + writePos + 1) + 1);
if (!finalResult)
{
    char* msg = "Error: unable to allocate memory\n";
    write(STDOUT_FILENO, msg, strlen(msg));
    free(result);
    return NULL;
}

strcpy(finalResult, result + writePos + 1);
free(result);
return finalResult;
}

char* hexDivide(const char* hex, uint64_t divisor) // like floor approximation
{
    if (divisor == 0)
    {
        return NULL;
    }

    if (!strcmp(hex, "0"))
    {
        return strdup("0");
    }

    size_t len = strlen(hex);
    char *result = malloc(len + 1);
    if (!result)
    {
        char* msg = "Error: unable to allocate memory\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        return NULL;
    }

    uint64_t remainder = 0;
    size_t resultPos = 0;
    for (size_t i = 0; i < len; ++i)
    {
        uint8_t digit = isdigit(hex[i]) ? hex[i] - '0' : hex[i] - 'a' + 10;
        remainder = remainder * 16 + digit;
    }

```

```

uint64_t quotient = remainder / divisor;
remainder = remainder % divisor;

if (quotient > 0 || resultPos > 0)
{
    result[resultPos++] = quotient < 10 ? quotient + '0' : quotient - 10 + 'a';
}
}

if (resultPos == 0)
{
    result[resultPos++] = '0';
}
result[resultPos] = 0;

return result;
}

```

main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <pthread.h>
#include <unistd.h>
#include <string.h>
#include <time.h>
#include "hexFuncs.h"

typedef struct
{
    char **numbers;
    uint64_t count;

    char *threadSum;
    uint64_t threadCount;
} ThreadData;

void *calcThreadPartialSum(void *arg)
{
    ThreadData *threadData = arg;
    char *threadSum = strdup("0");
    uint64_t threadCount = 0;

    for (uint64_t i = 0; i < threadData->count; ++i)
    {
        char *sumInThread = hexSum(threadSum, threadData->numbers[i]);
        free(threadSum);
        threadSum = sumInThread;
    }
}

```

```

        ++threadCount;
    }

    threadData->threadSum = threadSum;
    threadData->threadCount = threadCount;

    pthread_exit(NULL);
}

int main(int argc, char **argv)
{
    if (argc != 4)
    {
        char* msg = "Usage: ./main <threads_count> <memory_bytes> <file>\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        return 1;
    }

    struct timespec start, end;
    clock_gettime(CLOCK_MONOTONIC, &start);

    uint64_t threadsCount = strtoll(argv[1], NULL, 10);
    uint64_t memoryBytes = strtoll(argv[2], NULL, 10);

    FILE *file = fopen(argv[3], "r");
    if (!file)
    {
        char* msg = "Error: unable to open file\n";
        write(STDOUT_FILENO, msg, strlen(msg));
        exit(EXIT_FAILURE);
    }

    uint64_t bytesPerNumber = 32 + 1 + 8;
    uint64_t maxNumbersPerIteration = memoryBytes / bytesPerNumber;

    if (!maxNumbersPerIteration)
    {
        maxNumbersPerIteration = 1;
    }

    char buffer[64];
    char **allNumbers = malloc(sizeof(char*) * maxNumbersPerIteration);

    char *globalSum = strdup("0");
    uint64_t globalCount = 0;

    pthread_t *threads = malloc(sizeof(pthread_t) * threadsCount);

```



```

ThreadData *thread_data = malloc(sizeof(ThreadData) * threadsCount);

while (1)
{
    uint64_t totalCount = 0;

    while (totalCount < maxNumbersPerIteration && fgets(buffer, sizeof(buffer), file))
    {
        char *ptr = buffer;
        while (*ptr && *ptr != '\n')
        {
            ++ptr;
        }
        *ptr = 0;

        if (strlen(buffer))
        {
            allNumbers[totalCount] = strdup(buffer);
            ++totalCount;
        }
    }

    if (!totalCount)
    {
        break;
    }

    uint64_t numbersPerThread = totalCount / threadsCount;
    uint64_t remainder = totalCount % threadsCount;

    for (uint64_t i = 0; i < threadsCount; ++i)
    {
        uint64_t startIdx = i * numbersPerThread + (i > 0 ? remainder : 0);
        uint64_t count = numbersPerThread + (i == 0 ? remainder : 0);

        if (count)
        {
            thread_data[i].numbers = allNumbers + startIdx;
            thread_data[i].count = count;
            thread_data[i].threadSum = NULL;
            thread_data[i].threadCount = 0;

            pthread_create(&threads[i], NULL, calcThreadPartialSum, &thread_data[i]);
        }
    }

    for (uint64_t i = 0; i < threadsCount; ++i)

```

```

{
    uint64_t count = numbersPerThread + (i == 0 ? remainder : 0);

    if (count)
    {
        pthread_join(threads[i], NULL);
    }
}

for (uint64_t i = 0; i < threadsCount; ++i)
{
    uint64_t count = numbersPerThread + (i == 0 ? remainder : 0);

    if (count && thread_data[i].threadSum)
    {
        char* newSum = hexSum(globalSum, thread_data[i].threadSum);
        free(globalSum);
        globalSum = newSum;
        free(thread_data[i].threadSum);
        globalCount += thread_data[i].threadCount;
    }
}

for (uint64_t i = 0; i < totalCount; ++i)
{
    free(allNumbers[i]);
}
}

{
    char buf[64];
    sprintf(buf, "Amount of numbers: %lu\n", globalCount);
    write(STDOUT_FILENO, buf, strlen(buf));
}

{
    char buf[64];
    sprintf(buf, "Full sum: %s\n", globalSum);
    write(STDOUT_FILENO, buf, strlen(buf));
}

if (globalCount)
{
    char *average = hexDivide(globalSum, globalCount);
    {
        char buf[64];
        sprintf(buf, "Average floored hex value: %s\n", average);
    }
}

```

```

        write(STDOUT_FILENO, buf, strlen(buf));
    }
    free(average);
}

free(allNumbers);
free(globalSum);
free(threads);
free(thread_data);
fclose(file);

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed = (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1e9;
{
    char buf[64];
    sprintf(buf, "\nExecution time: %lf seconds\n", elapsed);
    write(STDOUT_FILENO, buf, strlen(buf));
}
{
    char buf[64];
    sprintf(buf, "Threads used: %lu\n", threadsCount);
    write(STDOUT_FILENO, buf, strlen(buf));
}

return 0;
}

```

Протокол работы программы

Тестирование:

```
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ ./main 1 1048576 /home/mrx/Desktop/OS/Lab2/test.txt
Amount of numbers: 1000000
Full sum: 79fcb733a4901222866385835293b284845ff
Average floored hex value: 7fe9ae855a07c656b29137b3ec5ab64e

Execution time: 1.367348 seconds
Threads used: 1
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ ./main 2 1048576 /home/mrx/Desktop/OS/Lab2/test.txt
Amount of numbers: 1000000
Full sum: 79fcb733a4901222866385835293b284845ff
Average floored hex value: 7fe9ae855a07c656b29137b3ec5ab64e

Execution time: 0.756378 seconds
Threads used: 2
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ ./main 4 1048576 /home/mrx/Desktop/OS/Lab2/test.txt
Amount of numbers: 1000000
Full sum: 79fcb733a4901222866385835293b284845ff
Average floored hex value: 7fe9ae855a07c656b29137b3ec5ab64e

Execution time: 0.454983 seconds
Threads used: 4
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ ./main 8 1048576 /home/mrx/Desktop/OS/Lab2/test.txt
Amount of numbers: 1000000
Full sum: 79fcb733a4901222866385835293b284845ff
Average floored hex value: 7fe9ae855a07c656b29137b3ec5ab64e

Execution time: 0.329845 seconds
Threads used: 8
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ ./main 16 1048576 /home/mrx/Desktop/OS/Lab2/test.txt
Amount of numbers: 1000000
Full sum: 79fcb733a4901222866385835293b284845ff
Average floored hex value: 7fe9ae855a07c656b29137b3ec5ab64e

Execution time: 0.286663 seconds
Threads used: 16
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ ./main 32 1048576 /home/mrx/Desktop/OS/Lab2/test.txt
Amount of numbers: 1000000
Full sum: 79fcb733a4901222866385835293b284845ff
Average floored hex value: 7fe9ae855a07c656b29137b3ec5ab64e

Execution time: 0.312535 seconds
Threads used: 32
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ ./main 100 1048576 /home/mrx/Desktop/OS/Lab2/test.txt
Amount of numbers: 1000000
Full sum: 79fcb733a4901222866385835293b284845ff
Average floored hex value: 7fe9ae855a07c656b29137b3ec5ab64e

Execution time: 0.341131 seconds
Threads used: 100
```

Strace:

```
mrx@pc:~/Desktop/OS/Lab2/cmake-build-debug$ strace -o strace_res.txt -f ./main 4 1048576
/home/mrx/Desktop/OS/Lab2/small_test.txt
2718601 execve("./main", ["/main", "4", "1048576", "/home/mrx/Desktop/OS/Lab2/small_...],
0x7ffea6d70eb0 /* 78 vars */) = 0
2718601 brk(NULL) = 0x649a85602000
2718601 mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x74741e988000
2718601 access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
2718601 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
2718601 fstat(3, {st_mode=S_IFREG|0644, st_size=79219, ...}) = 0
2718601 mmap(NULL, 79219, PROT_READ, MAP_PRIVATE, 3, 0) = 0x74741e974000
2718601 close(3) = 0
2718601 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
2718601 read(3, "\177ELF\2\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"..., 832) = 832
2718601 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
2718601 fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
2718601 pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784, 64) = 784
2718601 mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x74741e600000
2718601 mmap(0x74741e628000, 1605632, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x74741e628000
2718601 mmap(0x74741e7b0000, 323584, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x74741e7b0000
2718601 mmap(0x74741e7ff000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x74741e7ff000
2718601 mmap(0x74741e805000, 52624, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x74741e805000
2718601 close(3) = 0
2718601 mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
= 0x74741e971000
2718601 arch_prctl(ARCH_SET_FS, 0x74741e971740) = 0
2718601 set_tid_address(0x74741e971a10) = 2718601
2718601 set_robust_list(0x74741e971a20, 24) = 0
2718601 rseq(0x74741e972060, 0x20, 0, 0x53053053) = 0
2718601 mprotect(0x74741e7ff000, 16384, PROT_READ) = 0
2718601 mprotect(0x649a55a4d000, 4096, PROT_READ) = 0
2718601 mprotect(0x74741e9c6000, 8192, PROT_READ) = 0
2718601 prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0
2718601 munmap(0x74741e974000, 79219) = 0
2718601 getrandom("\xf6\xb2\xe4\xda\x02\xf5\xed\x64", 8, GRND_NONBLOCK) = 8
2718601 brk(NULL) = 0x649a85602000
2718601 brk(0x649a85623000) = 0x649a85623000
2718601 openat(AT_FDCWD, "/home/mrx/Desktop/OS/Lab2/small_test.txt", O_RDONLY) = 3
2718601 mmap(NULL, 204800, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
= 0x74741e93f000
2718601 fstat(3, {st_mode=S_IFREG|0664, st_size=230, ...}) = 0
2718601 read(3, "00000000000000000000000000000001"..., 4096) = 230
2718601 read(3, "", 4096) = 0
2718601 rt_sigaction(SIGRT_1, {sa_handler=0x74741e699530, sa_mask=[],
sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x74741e645330},
NULL, 8) = 0
2718601 rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0
2718601 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1,
0) = 0x74741ddff000
2718601 mprotect(0x74741de00000, 8388608, PROT_READ|PROT_WRITE) = 0
2718601 rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0
```

2718601

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x74741e5ff990, parent_tid=0x74741e5ff990, exit_signal=0, stack=0x74741ddff000, stack_size=0x7fff80, tls=0x74741e5ff6c0} => {parent_tid=[2718602]}, 88) = 2718602

2718601 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

2718602 rseq(0x74741e5ffe0, 0x20, 0, 0x53053053 <unfinished ...>

2718601 <... rt_sigprocmask resumed>NULL, 8) = 0

2718602 <... rseq resumed>) = 0

2718601 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0 <unfinished ...>

2718602 set_robust_list(0x74741e5ff9a0, 24 <unfinished ...>

2718601 <... mmap resumed>) = 0x74741d5fe000

2718602 <... set_robust_list resumed>) = 0

2718601 mprotect(0x74741d5ff000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

2718602 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

2718601 <... mprotect resumed>) = 0

2718602 <... rt_sigprocmask resumed>NULL, 8) = 0

2718601 rt_sigprocmask(SIG_BLOCK, ~[], <unfinished ...>

2718602 mmap(NULL, 134217728, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0 <unfinished ...>

2718601 <... rt_sigprocmask resumed>[], 8) = 0

2718602 <... mmap resumed>) = 0x747415400000

2718601

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x74741ddfe990, parent_tid=0x74741ddfe990, exit_signal=0, stack=0x74741d5fe000, stack_size=0x7fff80, tls=0x74741ddfe6c0} <unfinished ...>

2718602 munmap(0x747415400000, 46137344) = 0

2718601 <... clone3 resumed> => {parent_tid=[2718603]}, 88) = 2718603

2718603 rseq(0x74741ddfe0, 0x20, 0, 0x53053053 <unfinished ...>

2718602 munmap(0x74741c000000, 20971520 <unfinished ...>

2718601 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

2718603 <... rseq resumed>) = 0

2718601 <... rt_sigprocmask resumed>NULL, 8) = 0

2718602 <... munmap resumed>) = 0

2718603 set_robust_list(0x74741ddfe9a0, 24 <unfinished ...>

2718601 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0 <unfinished ...>

2718603 <... set_robust_list resumed>) = 0

2718602 mprotect(0x747418000000, 135168, PROT_READ|PROT_WRITE <unfinished ...>

2718601 <... mmap resumed>) = 0x74741cdfd000

2718603 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

2718601 mprotect(0x74741cdf000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

2718602 <... mprotect resumed>) = 0

2718603 <... rt_sigprocmask resumed>NULL, 8) = 0

2718601 <... mprotect resumed>) = 0

2718603 mmap(NULL, 134217728, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0 <unfinished ...>

2718601 rt_sigprocmask(SIG_BLOCK, ~[], <unfinished ...>

2718603 <... mmap resumed>) = 0x747410000000

2718601 <... rt_sigprocmask resumed>[], 8) = 0

2718602 openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC <unfinished ...>

2718601

clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID, child_tid=0x74741d5fd990, parent_tid=0x74741d5fd990, exit_signal=0, stack=0x74741cdfd000, stack_size=0x7fff80, tls=0x74741d5fd6c0} <unfinished ...>

```

2718603 munmap(0x747414000000, 67108864) = 0
2718602 <... openat resumed>          = 4
2718601 <... clone3 resumed> => {parent_tid=[2718604]}, 88) = 2718604
2718604 rseq(0x74741d5fdfe0, 0x20, 0, 0x53053053 <unfinished ...>
2718603 mprotect(0x747410000000, 135168, PROT_READ|PROT_WRITE <unfinished ...>
2718601 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>
2718604 <... rseq resumed>              = 0
2718602 fstat(4, <unfinished ...>
2718601 <... rt_sigprocmask resumed>NULL, 8) = 0
2718604 set_robust_list(0x74741d5fd9a0, 24 <unfinished ...>
2718603 <... mprotect resumed>          = 0
2718601 mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0
<unfinished ...>
2718604 <... set_robust_list resumed>) = 0
2718602 <... fstat resumed>{st_mode=S_IFREG|0644, st_size=79219, ...}) = 0
2718601 <... mmap resumed>              = 0x74741c5fc000
2718604 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>
2718603 futex(0x74741e9c8a08, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
2718601 mprotect(0x74741c5fd000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>
2718604 <... rt_sigprocmask resumed>NULL, 8) = 0
2718602 mmap(NULL, 79219, PROT_READ, MAP_PRIVATE, 4, 0 <unfinished ...>
2718604 mmap(0x747414000000, 67108864, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0
<unfinished ...>
2718601 <... mprotect resumed>          = 0
2718602 <... mmap resumed>              = 0x74741e974000
2718604 <... mmap resumed>              = 0x747414000000
2718601 futex(0x74741e9c8a58, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
2718602 close(4 <unfinished ...>
2718604 mprotect(0x747414000000, 135168, PROT_READ|PROT_WRITE <unfinished ...>
2718602 <... close resumed>            = 0
2718604 <... mprotect resumed>          = 0
2718604 futex(0x74741e9c8a08, FUTEX_WAIT_PRIVATE, 2, NULL <unfinished ...>
2718602 openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 4
2718602 read(4, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\0\0\0\0\0\0\0", 832) = 832
2718602 fstat(4, {st_mode=S_IFREG|0644, st_size=183024, ...}) = 0
2718602 mmap(NULL, 185256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 4, 0) =
0x74741e911000
2718602 mmap(0x74741e915000, 147456, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 4, 0x4000) = 0x74741e915000
2718602 mmap(0x74741e939000, 16384, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 4, 0x28000) = 0x74741e939000
2718602 mmap(0x74741e93d000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 4, 0x2b000) = 0x74741e93d000
2718602 close(4)                        = 0
2718602 mprotect(0x74741e93d000, 4096, PROT_READ) = 0
2718602 futex(0x74741e9c8a58, FUTEX_WAKE_PRIVATE, 1) = 1
2718601 <... futex resumed>              = 0
2718601 futex(0x74741e9c8a58, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>
2718602 munmap(0x74741e974000, 79219 <unfinished ...>
2718601 <... futex resumed>              = 0
2718601 rt_sigprocmask(SIG_BLOCK, ~[], <unfinished ...>
2718602 <... munmap resumed>            = 0
2718601 <... rt_sigprocmask resumed>~[], 8) = 0
2718602 futex(0x74741e9c8a08, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>
2718601
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLO
NE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x74741cdfc990, parent_tid=0x74741cdfc990, exit_signal=0, stack=0x74741c5fc000,

```

```
stack_size=0x7fff80, tls=0x74741cdfc6c0} <unfinished ...>  
2718602 <... futex resumed>) = 1  
2718603 <... futex resumed>) = 0  
2718603 futex(0x74741e9c8a08, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>  
2718602 futex(0x74741e93e230, FUTEX_WAKE_PRIVATE, 2147483647 <unfinished ...>  
2718604 <... futex resumed>) = 0  
2718603 <... futex resumed>) = 1  
2718601 <... clone3 resumed> => {parent_tid=[2718605]}, 88) = 2718605  
2718605 rseq(0x74741cdfcf0, 0x20, 0, 0x53053053 <unfinished ...>  
2718601 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>  
2718604 futex(0x74741e9c8a08, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>  
2718601 <... rt_sigprocmask resumed>NULL, 8) = 0  
2718605 <... rseq resumed>) = 0  
2718604 <... futex resumed>) = 0  
2718603 rt_sigprocmask(SIG_BLOCK, ~[RT_1], <unfinished ...>  
2718601 futex(0x74741e5ff990, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 2718602,  
NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>  
2718605 set_robust_list(0x74741cdfc9a0, 24 <unfinished ...>  
2718602 <... futex resumed>) = 0  
2718605 <... set_robust_list resumed>) = 0  
2718604 rt_sigprocmask(SIG_BLOCK, ~[RT_1], <unfinished ...>  
2718603 <... rt_sigprocmask resumed>NULL, 8) = 0  
2718605 rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>  
2718604 <... rt_sigprocmask resumed>NULL, 8) = 0  
2718605 <... rt_sigprocmask resumed>NULL, 8) = 0  
2718603 madvise(0x74741d5fe000, 8368128, MADV_DONTNEED <unfinished ...>  
2718604 madvise(0x74741cdffd000, 8368128, MADV_DONTNEED <unfinished ...>  
2718605 rt_sigprocmask(SIG_BLOCK, ~[RT_1], <unfinished ...>  
2718604 <... madvise resumed>) = 0  
2718603 <... madvise resumed>) = 0  
2718605 <... rt_sigprocmask resumed>NULL, 8) = 0  
2718604 exit(0 <unfinished ...>  
2718605 madvise(0x74741c5fc000, 8368128, MADV_DONTNEED <unfinished ...>  
2718603 exit(0 <unfinished ...>  
2718604 <... exit resumed>) = ?  
2718602 rt_sigprocmask(SIG_BLOCK, ~[RT_1], <unfinished ...>  
2718605 <... madvise resumed>) = 0  
2718603 <... exit resumed>) = ?  
2718605 exit(0 <unfinished ...>  
2718604 +++ exited with 0 +++  
2718603 +++ exited with 0 +++  
2718605 <... exit resumed>) = ?  
2718605 +++ exited with 0 +++  
2718602 <... rt_sigprocmask resumed>NULL, 8) = 0  
2718602 madvise(0x74741ddff000, 8368128, MADV_DONTNEED) = 0  
2718602 exit(0) = ?  
2718601 <... futex resumed>) = 0  
2718602 +++ exited with 0 +++  
2718601 write(1, "Amount of numbers: 7\n", 21) = 21  
2718601 write(1, "Full sum: 1e0000000000000000000000...", 44) = 44  
2718601 write(1, "Average floored hex value: 44924...", 60) = 60  
2718601 munmap(0x74741e93f000, 204800) = 0  
2718601 close(3) = 0  
2718601 write(1, "\nExecution time: 0.003316 second...", 34) = 34  
2718601 write(1, "Threads used: 4\n", 16) = 16  
2718601 exit_group(0) = ?  
2718601 +++ exited with 0 +++
```


Вывод

Во время выполнения лабораторной работы были изучены и использованы основные системные вызовы для работы с процессами в ОС Linux. Была создана программа, демонстрирующая создание потоков и вычисление среднего арифметического чисел из файла.

В ходе выполнения возникли проблемы с нахождением наиболее оптимального алгоритма для решения поставленной задачи.